

Keddeh Systems — VFS and executor fault qualification

Document ID: KEX-RD-VFS-FAULT-20261008-001 | Version: 0.1 | Date: 2026-10-08 Australia/Adelaide

Architecture originator: A. Keddeh. Analytical author/contributor: Codex, automated engineering and test drafting. Publisher: Keddeh Systems. Rights holder: A. Keddeh. Report licence: reserved rights; underlying repository code retains its existing MIT licence. Classification: owner engineering draft, synthetic fixtures only. Status: locally reproduced engineering evidence; no independent approval or production qualification.

Primary sector: SECTOR_EMPIRICAL_BENCHMARKS. Related sectors: SECTOR_SYSTEMS_ARCHITECTURE (bindings, transactions, backup); SECTOR_MATHEMATICAL_LATTICE (identity distinct from carrier equality); SECTOR_FOUNDATIONS (source custody). Exact publishing standard and DOCX template referenced by the publishing skill were not retrievable; this draft applies readable SKILL.md metadata requirements, without claiming exact layout/template conformance.

1. Abstract

A source-local audit found that content-digest deduplication collapsed path/source distinctions in the available BRAINK VFS implementation. The repair separates current path bindings from shared immutable carriers. Executor success accounting gains a dedicated lock; backups stage and verify committed-snapshot carriers rather than scanning a changing object directory. Thirty-eight test cases pass. This establishes finite local behaviours under controlled faults, not native KEX lineage authentication, physical RAM provisioning or universal failover safety.

2. Research question and hypothesis

Can shared carrier bytes preserve independent contextual bindings while concurrent operations, rollback and process interruption occur? Hypotheses: separate bindings preserve path-local provenance; SQLite commit gates prevent partially updated admission; verified snapshot staging prevents backup drift; successful callable accounting remains exact under concurrent invocations. Counterexamples would be mixed source fields, visible uncommitted paths, broken receipt chains, backup references without matching bytes or lost success counts.

3. Source context and architectural constraints

The available implementation is BRAINK-BETA-TEST, not the separately referenced capability runner or mesh CLI. Owner one-origin contextual lineage remains separate from numerical CPU ring levels. No authorization policy is substituted for native KEX derivation. A content digest identifies bytes; equal bytes do not imply equal path/source context. The new bindings table retains per-path source, predecessor, media type and creation timestamp. Legacy digest lookups and aggregate carrier edges remain diagnostic interfaces and must not be interpreted as contextual execution authority. Migration can preserve only metadata actually present in legacy records; it cannot recover overwritten distinctions automatically.

4. Method and controls

The suite runs existing domain/ring/dispatch/VFS checks plus fault-specific tests. Eight worker threads write 32 path-distinct records sharing one carrier; each source binding is checked and the serialized receipt chain is verified. Eight threads invoke 128 executor operations and verify every result and the count. Fault injection targets `os.replace`, receipt insertion and staged object corruption. A subprocess exits with code 73 before commit, followed by reopen and retry. A backup acquires its database snapshot, then sixteen controlled writes commit during object staging; archived database and carrier count remain those of the earlier snapshot. No live financial submissions, foreign runtimes or supplied document instructions are executed.

5. Results and executed evidence

All 38 test cases passed in the recorded run. Shared content produced one carrier and 32 distinct readable path/source bindings. Executor successes counted 128 exactly; denied and failed operations did not count. Receipt failure rolled back artifact, path and binding metadata. Failed replacement left no published artifact or temporary write file. Process exit left no committed path; retry admitted the retained bytes under a new valid binding. Corrupted carriers and modified receipts were detected. Backups excluded staged orphan objects, verified archived database/carrier digests, retained prior output on failure, and excluded sixteen post-snapshot commits. JUnit evidence and a source-pinned receipt are retained beside this report. No latency, energy or market valuation inference is made.

6. Critique and uncertainty

These are targeted finite tests on one host. Simulated exceptions and process exit are not physical power-loss certification. Immutable orphan bytes may remain; reclamation needs coordination with active writers and is not implemented. Counter locking does not make arbitrary callables thread-safe or freeze a mutable ring object. Existing bearer access remains a transport gate, not proof of native seed lineage. Coordinated hostile database/carrier substitution is outside these integrity checks. VFS digest-only lineage can aggregate edges across bindings; consumers must use path-local records for context. Historical provenance absent from old data remains unknown. No complete isolated-process security or external deployment is claimed.

7. Emergent learning and topology

The profound result is that deduplication can coexist with independent binding context when equality of physical bytes is kept below contextual identity. Transactional admission and immutable carrier retention also allow failed writes to be retried without portraying orphan bytes as executed or committed state. Backup correctness depends on the admitted snapshot, not everything found in the carrier directory. This aligns source-local learning with the owner's identity-versus-materialization distinction while clearly limiting the prototype to its available context fields. Next tests must cover historical binding revision/continuation semantics before treating this storage adapter as the native full-coordinate VFS.

8. Sources, reproduction and next task

Sources: owner BRAINK-BETA-TEST branch codex/function-configure-and-dispatch; vfs_server/store.py, backup.py, model.py; braink/runtime/executor.py; fault and executor tests; earlier function-configuration case study; owner number-line source register. Exact revision, digests, command and test count are in VERIFICATION.json. Reproduce with ``.venv/bin/python -m pytest -o addopts=" tests vfs_server/tests``. Static function maps are inventories, not dynamic coverage receipts. Next task: version contextual bindings and qualify continuation reuse, then integrate only with an acquired native capability-runner implementation.